Compte rendu CSC 8607: TP3
Réalisé par: Meriam Boukraa

Exercice 1:

Partie du code à compléter:

```python
# 1. Pipeline d'entraînement (acec Data Augmentation)
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(227),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=IMAGENET_MEAN,
        std=IMAGENET_STD
    ),
])

# 2. Pipeline de validation (déterministe)
val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(227),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=IMAGENET_MEAN,
        std=IMAGENET_STD
    ),
])
```

```python
def get_dataloaders(
    batch_size=256,
    n_workers=8,
    subset_size=None
):
    print("Chargement d'ImageNet...")

    train_dataset = ImageNet(
        root=data_dir,
        split="train",
        transform=train_transform
    )

    val_dataset = ImageNet(
        root=data_dir,
        split="val",
        transform=val_transform
    )
```

```python
train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=n_workers,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=n_workers,
    pin_memory=True
)
```

On peut même vérifier que mon script fonctionne sur le noeud de calcul:

```
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
$ python train_imagenet.py
Environnement prêt.
Appareil détecté : cuda
Chargement d'un petit échantillon d'ImageNet...
Chargement d'ImageNet...
Dimensions des images : torch.Size([4, 3, 227, 227])
Dimensions des étiquettes : torch.Size([4])
Nombre de classes dans ImageNet : 16
Test réussi : les images sont chargées.
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
```

Aussi j'ai fait un test qui crée un mini-lot d'images sur le GPU et applique une convolution similaire à la première couche d'AlexNet.

```
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
$ python -c "import torch; x=torch.randn(4,3,227,227,device='cuda'); conv=torch.nn.Conv2d(3,64,kernel_size=11,stride=4).
cuda(); y=conv(x); torch.cuda.synchronize(); print('Entrée :', x.shape); print('Sortie :', y.shape); print('Calcul effec
tué sur :', y.device); print('GPU :', torch.cuda.get_device_name(0))"
Entrée : torch.Size([4, 3, 227, 227])
Sortie : torch.Size([4, 64, 55, 55])
Calcul effectué sur : cuda:0
GPU : NVIDIA L4
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
```

Réponse aux questions:

1. Importance de la normalisation : La normalisation transforme les pixels selon (x'=(x-\mu)/\sigma), afin de centrer les données et d'homogénéiser leur échelle. Sans elle, l'apprentissage peut être moins stable ou plus lent. Les statistiques doivent être calculées uniquement sur l'entraînement pour éviter toute fuite d'information depuis la validation (data leakage).

2. Différence entre entraînement et validation : RandomResizedCrop et RandomHorizontalFlip créent des variantes des images d'entraînement pour limiter le surapprentissage. En validation, Resize et CenterCrop garantissent un prétraitement déterministe et une évaluation cohérente.

3. Exemple où le retournement horizontal est néfaste : Pour la reconnaissance de chiffres manuscrits, retourner horizontalement un chiffre peut modifier son apparence et le rendre ambigu, alors que son étiquette reste inchangée. Cela peut perturber l'apprentissage.

Exercice 2:

Etape 1:

Code de la classe MLP à compléter:

```python
class MLP(nn.Module):
    def __init__(self, input_size=3 * 227 * 227, hidden_size=256, num_classes=1000):
        super().__init__()

        self.classifier = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(inplace=True),
            nn.Linear(hidden_size, num_classes)
        )

    def forward(self, x):
        x = torch.flatten(x, 1)
        return self.classifier(x)


def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

Réponse aux questions:

1. Calcul du nombre de paramètres du MLP:

Le réseau possède trois couches linéaires. Chaque couche contient des poids et des biais.

Couche 1 : 154 587 entrées → 256 neurones

- Poids : (154587 x 256 = 39574272)
- Biais : (256)
- Total : (39574272 + 256 = 39574528)

Couche 2 : 256 neurones → 256 neurones

- Poids : (256 x 256 = 65536)
- Biais : (256)
- Total : (65536 + 256 = 65792)

Couche 3 : 256 neurones → 1 000 classes

- Poids : (256 x 1000 = 256000)
- Biais : (1,000)
- Total : (256000 + 1,000 = 257000)

Nombre total de paramètres :

Avec les biais :

39574528+65792+257000 = 39897320

Sans les biais :

39574272+65536+256000 = 39895808

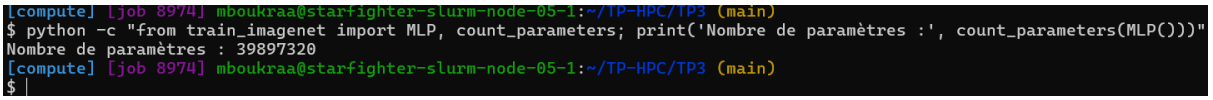
Couche concentrant plus de 99 % des paramètres :

La première couche contient :

39,574,528 / 39897320 x 100 = 99,19

C'est donc la première couche linéaire qui concentre plus de 99 % des paramètres.

Output dans le terminal:



2. Défaut structurel majeur du MLP : L'aplatissement (flatten) supprime la structure spatiale de l'image : le réseau ne tient pas naturellement compte de la proximité entre les pixels, contrairement à un réseau convolutif.

Etape 2:

1.Remplissage du tableau:

Couche	Taille entrée (C, H×W)		Hyperparamètres (Filtres/Neurones, K, S, P)				Taille sortie (C, H×W)		# Paramètres (avec biais)	Mémoire sortie (KB)
Conv1 + ReLU	3	227×227	64	11×11	S=4	P=2	64	56 x 56	23294	784 KB
MaxPool1	64	56 x 56	-	3×3	S=2	P=0	64	27 x 27	0	182,25 KB
Conv2 + ReLU	64	56 x 56	192	5×5	S=1	P=2	192	27 x 27	307392	546,75 KB
MaxPool2	192	27 x 27	-	3×3	S=2	P=0	192	13 x 13	0	126,75 KB
Conv3 + ReLU	192	13 x 13	384	3×3	S=1	P=1	384	13 x 13	663936	253,5 KB
Conv4 + ReLU	384	13 x 13	256	3×3	S=1	P=1	256	13 x 13	884992	169 KB
Conv5 + ReLU	256	13 x 13	256	3×3	S=1	P=1	256	13 x 13	590080	169 KB
MaxPool3	256	13 x 13	-	3×3	S=2	P=0	256	6 x 6	0	36 KB
Flatten + Dropout	256	6 x 6	p = 0.5				9216(1D)		0	36 KB
Linear1 + ReLU + Dropout	9216		4096 neurones (p = 0.5)				4096		37752832	16 KB
Linear2 + ReLU	4096		4096 neurones				4096		16781312	16 KB

Linear3 (Sortie)	4096	1000 classes	1000	4097000	3.91 KB

2. Les paramètres se concentrent principalement dans les couches linéaires, surtout Linear1, qui possède 37752832 paramètres. À l'inverse, la mémoire des activations est principalement utilisée par les premières couches convolutives, notamment Conv1 et Conv2, car leurs cartes de caractéristiques sont volumineuses.

3. Comparaison avec le MLP : Le MLP possède 39897320 paramètres, contre 61 100 840 pour AlexNet. Pourtant, le MLP utilise moins de mémoire pour ses activations internes : ses couches cachées contiennent seulement 256 neurones, tandis qu'AlexNet produit de grandes cartes de caractéristiques intermédiaires. La mémoire des activations dépend donc aussi des dimensions des sorties de chaque couche, et pas seulement du nombre de paramètres.

Etape 3:

La partie du code à compléter:

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),

            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),

            nn.MaxPool2d(kernel_size=3, stride=2),
        )

        self.classifier = nn.Sequential(
            nn.Dropout(p=0.5),
            nn.Linear(256 * 6 * 6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(p=0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Linear(4096, num_classes),
        )
```

La fonction count_parameters() donne 39897320 paramètres pour le MLP et 61100840 paramètres pour AlexNet. Ces résultats confirment les calculs théoriques effectués précédemment.

```
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
$ python -c "from train_imagenet import MLP, AlexNet, count_parameters; print('MLP :', count_parameters(MLP())); print('
AlexNet :', count_parameters(AlexNet()))"
MLP : 39897320
AlexNet : 61100840
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
$
```

Bonus: J'ai même fait un test sur GPU qui confirme qu'AlexNet accepte une entrée de dimensions 1 × 3 × 227 × 227 et produit une sortie de dimensions 1 × 1000, correspondant aux scores des 1 000 classes d'ImageNet. Le modèle fonctionne correctement sur le GPU NVIDIA L4.

Exercice 3:

Code de la fonction evaluate:

```python
@torch.no_grad()
def evaluate(loader, model, criterion):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0

    for images, labels in tqdm(loader, desc="Eval", leave=False):
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        outputs = model(images)
        loss = criterion(outputs, labels)

        running_loss += loss.item() * images.size(0)
        _, predicted = outputs.max(1)

        correct += (predicted == labels).sum().item()
        total += labels.size(0)

    return running_loss / total, correct / total
```

Code de la fonction evaluate: J'ai pris une capture que des parties à compléter:

```python
for epoch in range(num_epochs):
    model.train()
```

```python
# --- EARLY STOPPING ---
if val_loss < best_val_loss:
    best_val_loss = val_loss
    early_stop_counter = 0
else:
    early_stop_counter += 1

if early_stop_counter >= patience:
    print("Early stopping déclenché !")
    break
```

Après, j'ai lancé successivement un entraînement de 10 époques sur le sous-ensemble (subset_size=20000) pour mlp puis pour alexnet:
MLP:

```
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
$ python -c "from train_imagenet import train_model; train_model(model_name='mlp', num_epochs=10, subset_size=20000)"
Chargement d'ImageNet...
/mnt/hdd/homes/mboukraa/miniforge3/envs/deeplearning/lib/python3.10/site-packages/torch/utils/data/dataloader.py:431: UserWarning: This DataLoader will create 8 worker processes in total. Our suggested max number of worker in current system is 1, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
  self.check_worker_number_rationality()
Modèle mlp initialisé avec 39,897,320 paramètres.
Epoch 1/10:   0%|                                                                                | 0/79 [00:00<?, ?it/s]/mnt/hdd/homes/mboukraa/miniforge3/envs/deeplearning/lib/python3.10/site-packages/torch/utils/data/dataloader.py:437: UserWarning: This DataLoader will create 8 worker processes in total. Our suggested max number of worker in current system is 1, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
  self.check_worker_number_rationality()
Epoch 1/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:25<00:00,  1.09s/it]
Epoch [1/10] | Train Loss: 7.1561 (Acc: 0.0010) | Val Loss: 7.0125 (Acc: 0.0022)
Epoch 2/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.00s/it]
Epoch [2/10] | Train Loss: 6.9602 (Acc: 0.0013) | Val Loss: 6.9470 (Acc: 0.0026)
Epoch 3/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.01s/it]
Epoch [3/10] | Train Loss: 6.9253 (Acc: 0.0022) | Val Loss: 6.9436 (Acc: 0.0028)
Epoch 4/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.00s/it]
Epoch [4/10] | Train Loss: 6.9112 (Acc: 0.0029) | Val Loss: 6.9328 (Acc: 0.0022)
Epoch 5/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.01s/it]
Epoch [5/10] | Train Loss: 6.8874 (Acc: 0.0039) | Val Loss: 6.9128 (Acc: 0.0028)
Epoch 6/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.01s/it]
Epoch [6/10] | Train Loss: 6.8599 (Acc: 0.0041) | Val Loss: 6.8936 (Acc: 0.0032)
Epoch 7/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.01s/it]
Epoch [7/10] | Train Loss: 6.8356 (Acc: 0.0046) | Val Loss: 6.8789 (Acc: 0.0030)
Epoch 8/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.01s/it]
Epoch [8/10] | Train Loss: 6.8183 (Acc: 0.0055) | Val Loss: 6.8745 (Acc: 0.0040)
Epoch 9/10: 100%|████████████████████████████████████████████████████████████████████| 79/79 [01:20<00:00,  1.02s/it]
Epoch [9/10] | Train Loss: 6.7985 (Acc: 0.0051) | Val Loss: 6.8713 (Acc: 0.0040)
Epoch 10/10: 100%|███████████████████████████████████████████████████████████████████| 79/79 [01:19<00:00,  1.01s/it]
Epoch [10/10] | Train Loss: 6.7757 (Acc: 0.0067) | Val Loss: 6.8573 (Acc: 0.0056)
[compute] [job 8974] mboukraa@starfighter-slurm-node-05-1:~/TP-HPC/TP3 (main)
$
```

alexnet:

```
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
$ python -c "from train_imagenet import train_model; train_model(model_name='alexnet', batch_size=64, num_epochs=10, subset_size=20000)"
Chargement d'ImageNet...
Modèle alexnet initialisé avec 61,100,840 paramètres.
Epoch 1/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:22<00:00,  3.82it/s]
Epoch [1/10] | Train Loss: 6.9084 (Acc: 0.0006) | Val Loss: 6.9075 (Acc: 0.0014)
Epoch 2/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:19<00:00,  3.95it/s]
Epoch [2/10] | Train Loss: 6.8895 (Acc: 0.0014) | Val Loss: 6.8565 (Acc: 0.0020)
Epoch 3/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:20<00:00,  3.89it/s]
Epoch [3/10] | Train Loss: 6.8078 (Acc: 0.0016) | Val Loss: 6.8091 (Acc: 0.0028)
Epoch 4/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:18<00:00,  3.98it/s]
Epoch [4/10] | Train Loss: 6.7619 (Acc: 0.0023) | Val Loss: 6.7919 (Acc: 0.0022)
Epoch 5/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:19<00:00,  3.94it/s]
Epoch [5/10] | Train Loss: 6.7206 (Acc: 0.0027) | Val Loss: 6.7247 (Acc: 0.0030)
Epoch 6/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:20<00:00,  3.91it/s]
Epoch [6/10] | Train Loss: 6.6409 (Acc: 0.0043) | Val Loss: 6.6629 (Acc: 0.0056)
Epoch 7/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:20<00:00,  3.89it/s]
Epoch [7/10] | Train Loss: 6.5511 (Acc: 0.0052) | Val Loss: 6.5647 (Acc: 0.0076)
Epoch 8/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:20<00:00,  3.89it/s]
Epoch [8/10] | Train Loss: 6.4538 (Acc: 0.0070) | Val Loss: 6.5088 (Acc: 0.0100)
Epoch 9/10: 100%|████████████████████████████████████████████████████████████████| 313/313 [01:20<00:00,  3.90it/s]
Epoch [9/10] | Train Loss: 6.3410 (Acc: 0.0100) | Val Loss: 6.3968 (Acc: 0.0122)
Epoch 10/10: 100%|███████████████████████████████████████████████████████████████| 313/313 [01:21<00:00,  3.86it/s]
Epoch [10/10] | Train Loss: 6.2238 (Acc: 0.0133) | Val Loss: 6.2678 (Acc: 0.0136)
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
```

Réponses aux questions:

1. La méthode model.train() place le modèle en mode entraînement. Dans AlexNet, elle active notamment les couches nn.Dropout(p=0.5), qui désactivent aléatoirement 50 % des activations concernées à chaque passage. Cette opération permet de limiter le surapprentissage en évitant que le réseau dépende trop fortement de certaines connexions.

À l'inverse, model.eval() place le modèle en mode évaluation et désactive le Dropout : toutes les activations sont utilisées normalement. Cela permet d'obtenir des prédictions stables et d'évaluer les performances du modèle sans perturbation aléatoire liée au Dropout.

Si l'on oubliait model.eval() pendant l'évaluation, le Dropout resterait actif. Les prédictions et les métriques de validation pourraient alors varier d'un passage à l'autre, et les performances mesurées pourraient être artificiellement dégradées.

Il est donc important d'utiliser model.train() pendant l'entraînement et model.eval() pendant la validation. À noter que model.eval() ne désactive pas à lui seul le calcul des gradients ; on utilise généralement aussi torch.no_grad() pour économiser la mémoire et accélérer l'évaluation.

2. Pour un problème de classification avec 1 000 classes, un modèle qui prédit au hasard a théoriquement une accuracy de 0,1 %, car il a une chance sur 1 000 de trouver la bonne classe.

La CrossEntropyLoss théorique est d'environ 6,9078.

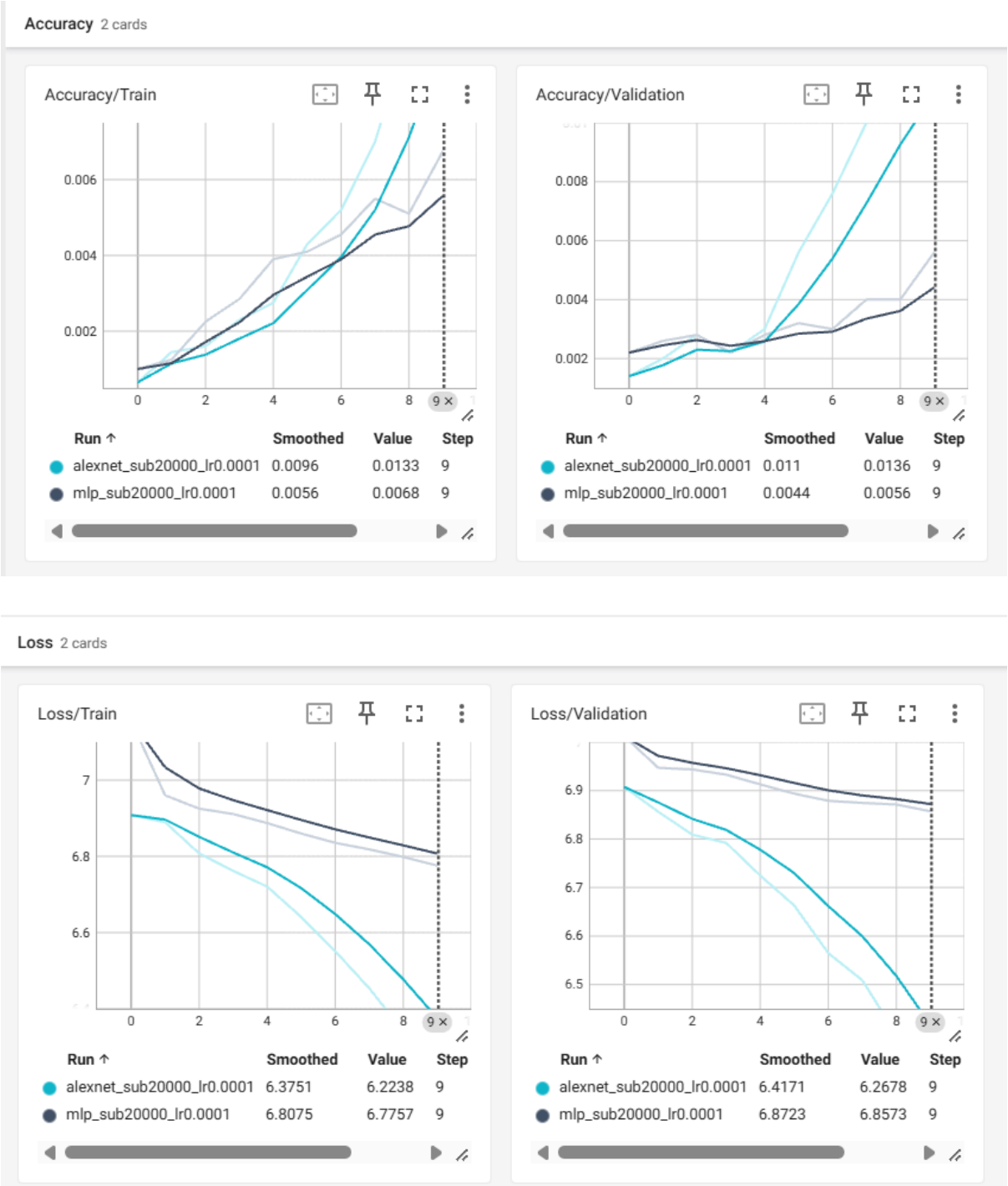
Les premières époques de nos expériences sont cohérentes avec ces valeurs. Pour le MLP, à la première époque, la loss d'entraînement est de 7,1561 et l'accuracy de 0,10 %. Pour AlexNet, la loss d'entraînement est de 6,9084 et l'accuracy de 0,06 %, tandis que la loss de validation est de 6,9075 et l'accuracy de 0,14 %.

Ces résultats sont proches des valeurs théoriques attendues au début de l'apprentissage. Les petites différences sont normales, car les poids des modèles sont initialisés aléatoirement et les prédictions ne sont pas parfaitement uniformes.

Après 10 époques, AlexNet montre une amélioration : sa loss d'entraînement diminue de 6,9084 à 6,2238 et son accuracy de validation augmente de 0,14 % à 1,36 %. Cela montre que le modèle commence à apprendre à distinguer les images, même si ses performances restent encore faibles.

3.

Les courbes de Tensorflow:



Comparaison:
Les courbes TensorBoard montrent qu'AlexNet obtient de meilleures performances que le MLP sur le sous-ensemble de 20 000 images d'ImageNet. Après 10 époques, AlexNet atteint une accuracy de validation de 1,36 %, contre 0,56 % pour le MLP. Sa loss de validation est également plus faible (6,2678 contre 6,8573). Cette différence peut s'expliquer par l'architecture convolutive d'AlexNet, mieux adaptée à l'extraction de caractéristiques spatiales des images, tandis que le MLP traite les pixels sous forme de vecteur. Concernant le surapprentissage, les losses d'entraînement et de validation diminuent pour les deux modèles. Aucun signe évident d'overfitting n'est donc observé sur les 10 époques étudiées.

Il n'est pas possible de déterminer lequel surapprend le plus vite à partir de ces seuls résultats.

Etape 2:

La variante plus profonde d'AlexNet a été créée avec succès, avec l'ajout de trois couches de convolution après Conv5:

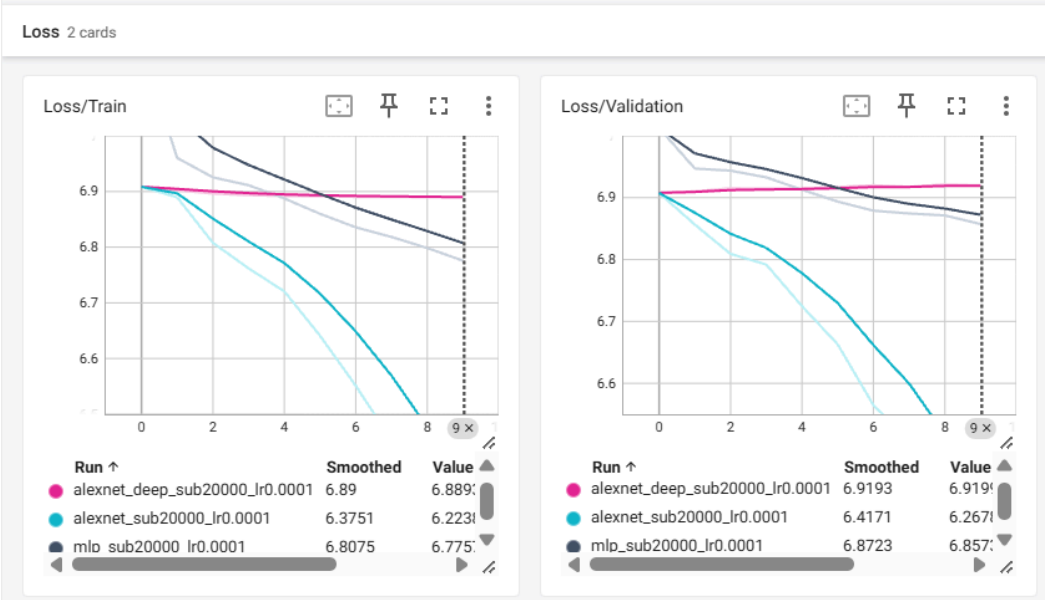
```
AlexNetDeep ajouté avec succès.
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
$ python -m py_compile train_imagenet.py
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
$
```

Après j'ai lancé un test rapide sur 10 époques avec subset_size=20000:

```
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
$ python -c 'from train_imagenet import train_model; train_model(model_name="alexnet_deep", batch_size=64, lr=1e-4, num_
epochs=10, patience=10, subset_size=20000)'
Chargement d'ImageNet...
Modèle alexnet_deep initialisé avec 62,871,080 paramètres.
Epoch 1/10: 100%|| 313/313 [01:17<00:00,  4.03it/s]
Epoch [1/10] | Train Loss: 6.9085 (Acc: 0.0009) | Val Loss: 6.9078 (Acc: 0.0006)
Epoch 2/10: 100%|| 313/313 [01:19<00:00,  3.91it/s]
Epoch [2/10] | Train Loss: 6.9023 (Acc: 0.0013) | Val Loss: 6.9101 (Acc: 0.0008)
Epoch 3/10: 100%|| 313/313 [01:19<00:00,  3.95it/s]
Epoch [3/10] | Train Loss: 6.8961 (Acc: 0.0014) | Val Loss: 6.9151 (Acc: 0.0004)
Epoch 4/10: 100%|| 313/313 [01:18<00:00,  4.01it/s]
Epoch [4/10] | Train Loss: 6.8933 (Acc: 0.0015) | Val Loss: 6.9144 (Acc: 0.0006)
Epoch 5/10: 100%|| 313/313 [01:20<00:00,  3.87it/s]
Epoch [5/10] | Train Loss: 6.8919 (Acc: 0.0011) | Val Loss: 6.9147 (Acc: 0.0008)
Epoch 6/10: 100%|| 313/313 [01:20<00:00,  3.88it/s]
Epoch [6/10] | Train Loss: 6.8910 (Acc: 0.0015) | Val Loss: 6.9171 (Acc: 0.0008)
Epoch 7/10: 100%|| 313/313 [01:19<00:00,  3.93it/s]
Epoch [7/10] | Train Loss: 6.8903 (Acc: 0.0018) | Val Loss: 6.9201 (Acc: 0.0008)
Epoch 8/10: 100%|| 313/313 [01:19<00:00,  3.95it/s]
Epoch [8/10] | Train Loss: 6.8901 (Acc: 0.0016) | Val Loss: 6.9174 (Acc: 0.0008)
Epoch 9/10: 100%|| 313/313 [01:20<00:00,  3.87it/s]
Epoch [9/10] | Train Loss: 6.8895 (Acc: 0.0013) | Val Loss: 6.9214 (Acc: 0.0008)
Epoch 10/10: 100%|| 313/313 [01:20<00:00,  3.87it/s]
Epoch [10/10] | Train Loss: 6.8893 (Acc: 0.0018) | Val Loss: 6.9199 (Acc: 0.0008)
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
```

Voici les nouvelles courbes:





AlexNetDeep obtient de moins bonnes performances qu'AlexNet standard : son accuracy de validation est de 0,08 %, contre 1,36 %, et sa loss diminue très peu pendant l'entraînement. Cela indique principalement un sous-apprentissage (*underfitting*), plutôt qu'un surapprentissage, car le modèle apprend difficilement même sur les données d'entraînement.

Exercice 4:

Changement du bloc if __name__ == "__main__":

Lancement de AlexNet:

```
[compute] [job 8996] mboukraa@starfighter-slurm-node-02-1:~/TP-HPC/TP3 (main)
$ cat > run_imagenet.sh <<'EOF'
#!/bin/bash
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=24G
#SBATCH --time=04:00:00
#SBATCH -J alexnet-full
#SBATCH -o logs/%x-%j.out
#SBATCH -e logs/%x-%j.err

set -euo pipefail

source ~/miniforge3/etc/profile.d/conda.sh
conda activate deeplearning

echo "Début du job $SLURM_JOB_ID sur le nœud $SLURM_NODELIST"
date
nvidia-smi

python -u train_imagenet.py

echo "Fin de l'entraînement"
date
EOF
```

Le job d'entraînement d'AlexNet sur l'ensemble du jeu de données ImageNet a été soumis avec succès à l'aide de Slurm. Il a reçu l'identifiant 9002 et s'exécute sur la partition GPU, sur le nœud starfighter-slurm-node-02-1. La commande squeue indique l'état R (Running), ce qui confirme que le job est en cours d'exécution:

```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP3 (main)
$ sbatch --account=csc_8607 --qos=normal run_imagenet.sh
Submitted batch job 9040
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP3 (main)
$ squeue -u "$USER"
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
              9040       gpu alexnet- mboukraa  R       0:13      1 starfighter-slurm-node-07-1
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP3 (main)
$
```

Réponse aux questions:

1.

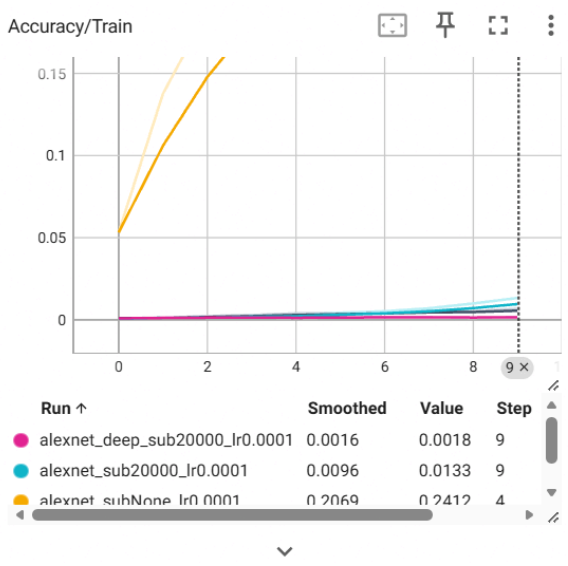
L'entraînement d'une époque sur les 1,28 million d'images d'ImageNet prend 2 h 05 min 07 s sur le GPU utilisé. La validation s'effectue ensuite sur le jeu de validation et ajoute plusieurs minutes. La durée totale estimée d'une époque est donc d'environ 2 h 11 min. Sans Early Stopping, 100 époques nécessiteraient environ 218 heures, soit un peu plus de 9 jours, en supposant une durée constante:

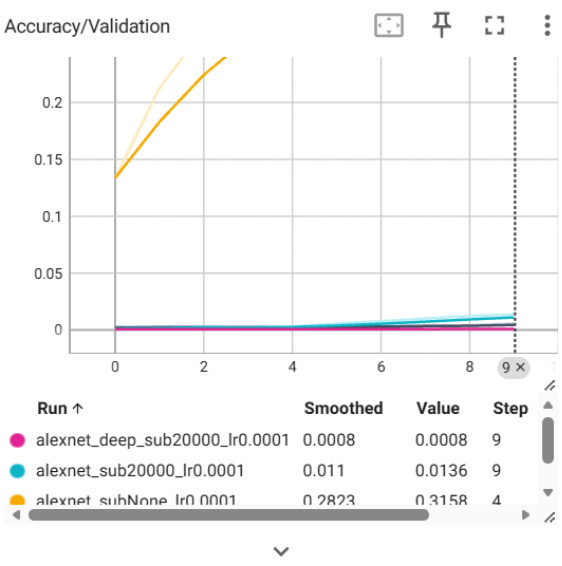
```
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50:  88%|████████▊ | 17671/20019 [1:50:24<13:54,  2.81it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50:  93%|█████████▎| 18702/20019 [1:56:49<08:10,  2.68it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50:  96%|█████████▌| 19245/20019 [2:00:14<04:45,  2.71it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50:  98%|█████████▊| 19622/20019 [2:02:37<02:27,  2.69it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50:  99%|█████████▉| 19791/20019 [2:03:42<01:32,  2.47it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50: 100%|██████████| 20019/20019 [2:05:07<00:00,  2.51it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-
07-1:~/TP-HPC/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50: 100%|██████████| 20019/20019 [2:05:07<00:00,  2.67it/s]
Eval:   9%|▉         | 69/782 [00:30<05:12,  2.28it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-07-1:~/TP-HPC
/TP3 (main)
$ tail -n 20 logs/alexnet-full-9040.err
Epoch 1/50: 100%|██████████| 20019/20019 [2:05:07<00:00,  2.67it/s]
Epoch 2/50:  19%|▏         | 3767/20019 [23:00<1:45:10,  2.58it/s][compute] [job 9040] mboukraa@starfighter-slurm-node-0
7-1:~/TP-HPC/TP3 (main)
$
```

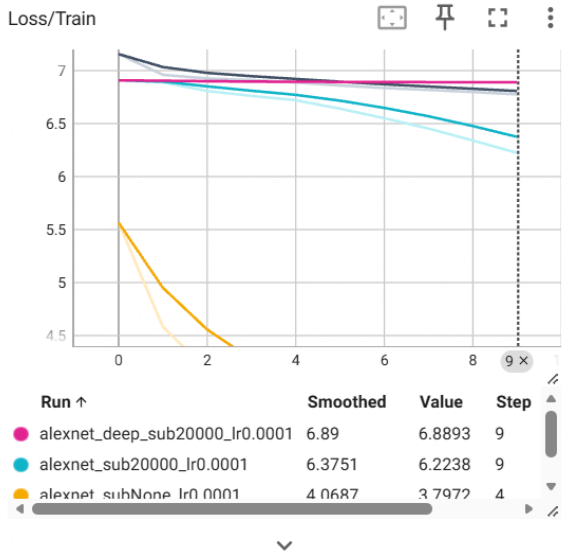
2. Voici une capture des 5 premières époches:

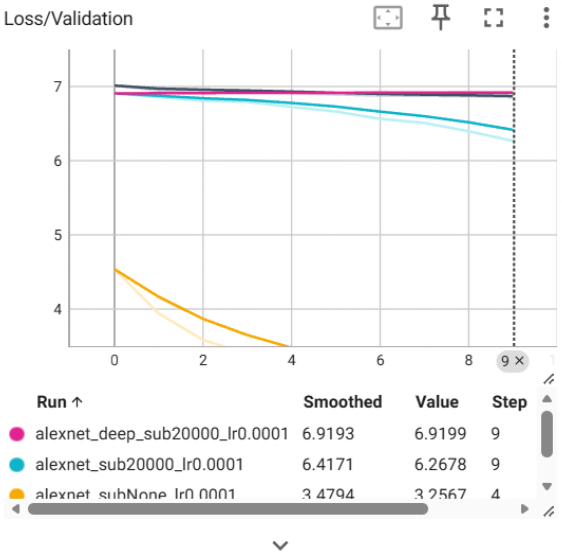
```
[controller] mboukraa@arcadia-slurm-controller:~/TP-HPC/TP3 (main)
$ tail -n 10 logs/alexnet-full-9040.err
Epoch 1/50: 100%|██████████| 20019/20019 [2:05:07<00:00,  2.67it/s]
Epoch 2/50: 100%|██████████| 20019/20019 [2:04:44<00:00,  2.67it/s]
Epoch 3/50: 100%|██████████| 20019/20019 [2:04:07<00:00,  2.69it/s]
Epoch 4/50: 100%|██████████| 20019/20019 [2:03:46<00:00,  2.70it/s]
Epoch 5/50: 100%|██████████| 20019/20019 [2:04:42<00:00,  2.68it/s]
```

Les courbes Tensorflow:









Comparaison:

Epoque	Loss/Train	Loss/Validation	Accuracy/Train	Accuracy/Validation
1	5,5659	4,5376	5,32%	13,35%
2	4,5823	3,9406	13,76%	21,25%
3	4,1792	3,5847	18,79%	26,37%
4	3,9466	3,3926	21,93%	29,47%
5	3,7972	3,2567	24,12%	31,58%

Au cours des cinq premières époques, la perte d'entraînement diminue de 5,5659 à 3,7972, tandis que la perte de validation diminue de 4,5376 à 3,2567. En parallèle, l'accuracy

d'entraînement progresse de 5,32 % à 24,12 %, et l'accuracy de validation de 13,35 % à 31,58 %. Ces résultats montrent que le modèle apprend progressivement à classifier les images.

La perte de validation reste inférieure à la perte d'entraînement. Ce phénomène peut notamment s'expliquer par la Data Augmentation, qui rend les exemples d'entraînement plus variés, et par le Dropout, actif pendant l'entraînement mais désactivé pendant la validation.

Comparaison avec les 20 000 images de l'exercice 3:

Sur le dataset complet, l'écart entre les pertes à la cinquième époque est :

$$3,7972 - 3,2567 = 0,5405$$

L'explication générale est que le dataset complet fournit beaucoup plus d'exemples pour apprendre à généraliser. La Data Augmentation introduit également des variations dans les images d'entraînement, ce qui peut limiter le sur-apprentissage.

3.Pourquoi la perte de validation est parfois plus faible:

- Data Augmentation : les images d'entraînement sont transformées aléatoirement, contrairement aux images de validation.
- Dropout : il est actif pendant l'entraînement pour désactiver aléatoirement certains neurones, mais désactivé pendant la validation.

Ces deux mécanismes peuvent rendre l'entraînement plus difficile que la validation et expliquer pourquoi Loss/Validation est parfois inférieure à Loss/Train.